Algorithmic Paradigms

Greedy: Build up a solution incrementally, optimize some local criterion.

Divide-and-conquer: Break up a problem into two sub-problems, solve each sub-problem independently, and combine solution to sub-problems to form a solution to the original problem.

Dynamic programming: Break up a problem into a series of overlapping sub-problems, and build up solutions to larger and larger sub-problems.

Matrix Chain Multiplication

Dynamic Programming is a general algorithm design paradigm.

Example: Matrix Multiplication.

- **C** = **A***B
- **A** is *m* × *n* and **B** is *n* × *p*

$$C[i, j] = \sum_{k=1}^{n} A[i, k] * B[k, j]$$

- Time Complexity: **O**(mnp)

Matrix Chain Multiplication

- Compute $A = A_1 * A_2 * \ldots * A_n$
- Size of A_i is $d_{i-1} \times d_i$
- **Problem**: How to parenthesize?

Example

- B is 3×100
- C is 100×5
- D is 5×5

(B*C)*D takes 1500 + 75 = 1575 operations

B*(C*D) takes 1500 + 2500 = 4000 operations

Greedy Approach #1

Idea #1: repeatedly select the product that uses the **most** operations.

Counter-example:

A is 10×5
B is 5×10
C is 10×5
D is 5×10

- Greedy idea #1 gives (A*B)*(C*D), which takes 500+1000+500 = 2000 operations
- But, A*((B*C)*D) takes 500+250+250 = 1000 operations

Greedy Approach #2

Idea #2: repeatedly select the product that uses the **fewest** operations.

Counter-example:

A is 101 × 11
B is 11 × 9
C is 9 × 100
D is 100 × 99

- Greedy idea #2 gives A*((B*C)*D)), which takes 109989+9900+108900 = 228789 operations
- But, (A*B)*(C*D) takes 9999+89991+89100=189090 operations

Recursive Approach

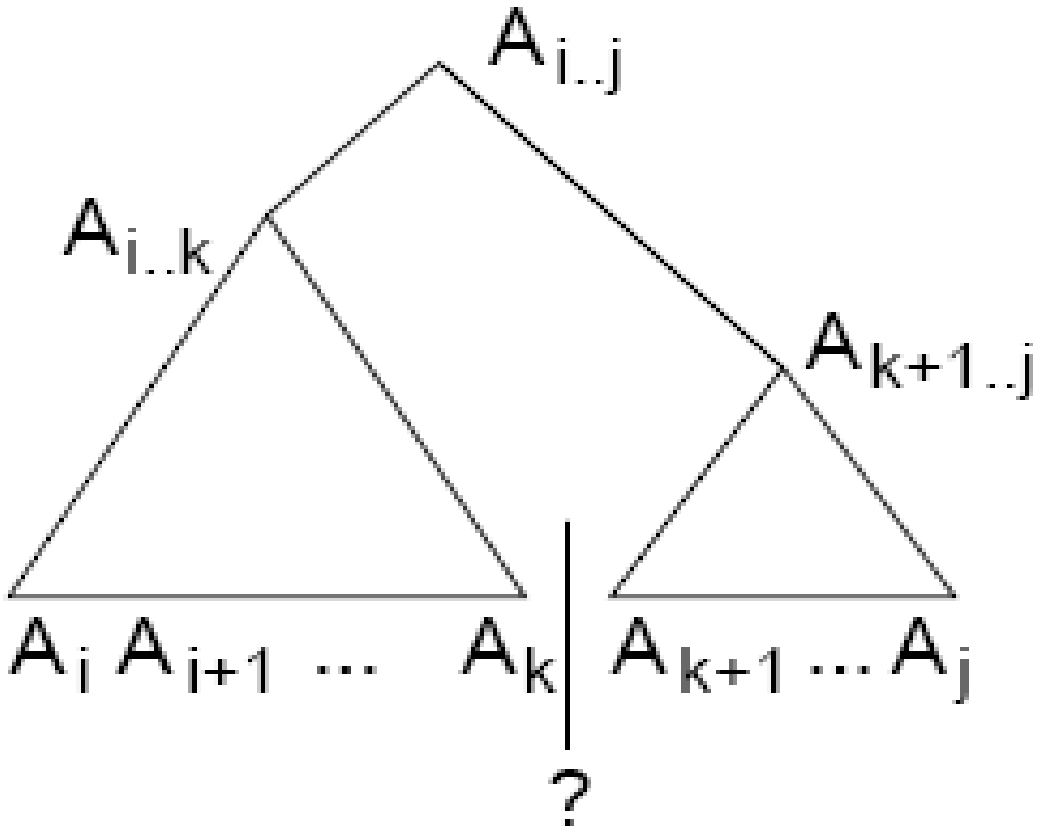
Define subproblems:

- Find the best parenthesization of $A_i * A_{i+1} * \ldots * A_j$.
- Let $N_{i,j}$ denote the number of operations done by this subproblem.
- The optimal solution for the whole problem is $N_{1,n}$.

Subproblem optimality: The optimal solution can be defined in terms of optimal subproblems

- There has to be a final multiplication (root of the expression tree) for the optimal solution.
- Let the final multiplication is at index i : $(A_1 * \ldots * A_k) * (A_{k+1} * \ldots * A_n)$.
- Then the optimal solution $N_{1,n}$ is the sum of two optimal subproblems, $N_{1,k}$ and $N_{k+1,n}$, plus the time for the last multiplication.

Recursive Approach



$$N_{i,j} = \min_{i \le k < j} \{ N_{i,k} + N_{k+1,j} + d_{i-1} d_k d_j \}$$

Longest Common Subsequence (LCS)

If $|X| = m$, $|Y| = n$, then there are 2^m subsequences of X; we need to compare each with Y (n comparisons).

The running time of the brute-force algorithm is $O(n2^m)$

Note that the LCS problem has *optimal substructure*: solutions of subproblems are parts of the final solution.

LCS Algorithm

First, find the length of LCS. Then modify the algorithm to find LCS itself.

Define X_i, Y_j to be the prefixes of X and Y of length *i* and *j*, respectively

Define c[i,j] to be the length of LCS of X_i and Y_j

Then the length of LCS of X and Y will be c[m,n]

LCS recursive solution

$$c[i, j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

We start with $i = j = 0$ (empty substrings of X and Y)

Since X_0 and Y_0 are empty strings, their LCS is always empty (c[0,0] = 0)

LCS of empty string and any other string is empty, so for every i and j: c[0, j] = c[i,0] = 0

LCS recursive solution

$$c[i, j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

In order to calculate c[i,j], we consider two cases:

- **First case** x[i]=y[j]: one more symbol in strings X and Y matches, so the length of LCS X_i and Y_j equals to the length of LCS of smaller strings X_{i-1} and Y_{i-1} , plus 1
- **Second case** x[i] ≠ y[j] : As symbols do not match, our solution is not improved, and the length of LCS(X_i , Y_j) is the same as before (i.e. maximum of LCS(X_i, Y_{j-1}) and LCS(X_{i-1},Y_j)

LCS Example

Let:
X = ABCB
Y = BDCAB

Q. What is the Longest Common Subsequence of X and Y?

LCS(X, Y) = BCB

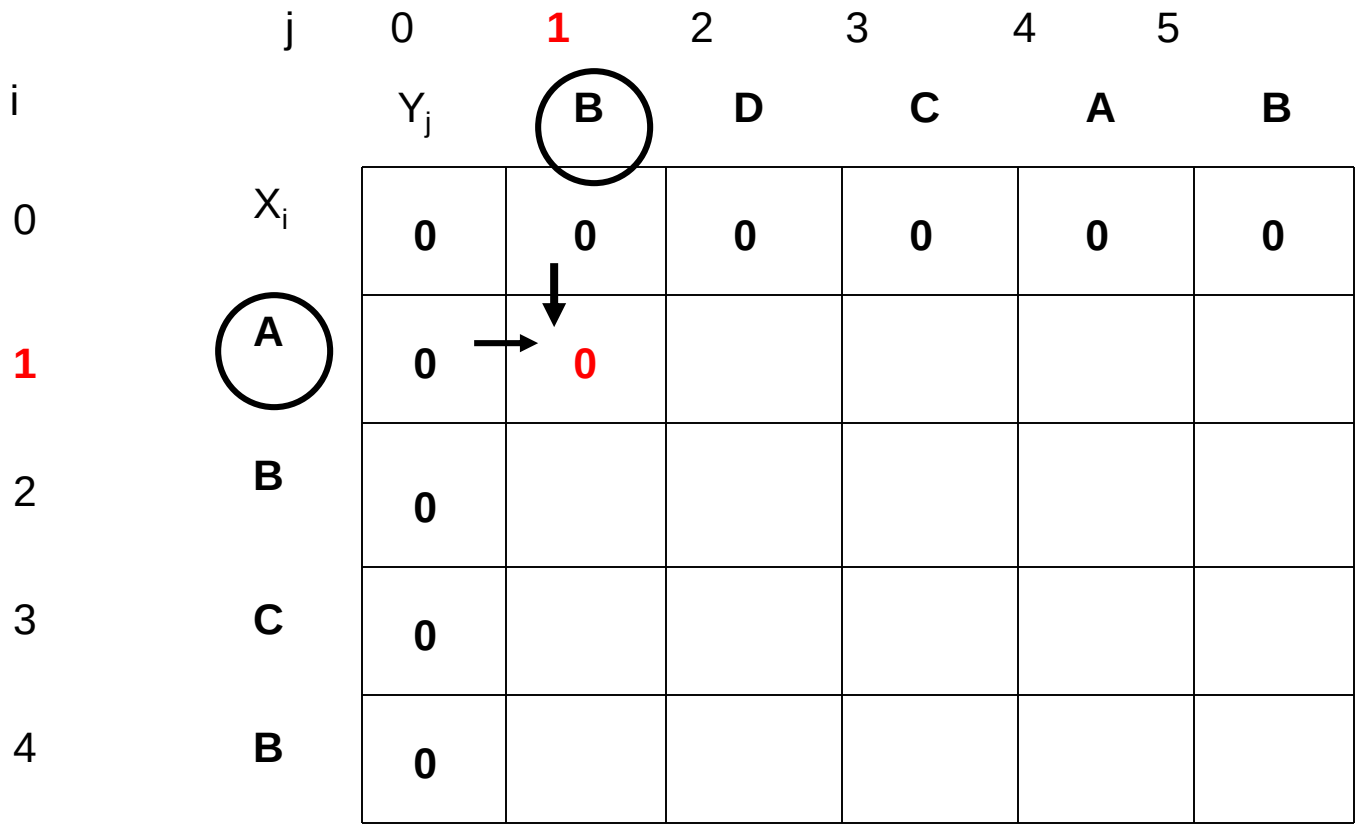
X = A **B** **C** **B**
Y = **B** D **C** A **B**

i	j	0	1	2	3	4	5
		Y_j	**B**	**D**	**C**	**A**	**B**
0	X_i						
1	**A**						
2	**B**						
3	**C**						
4	**B**						

X = ABCB; m = |X| = 4
Y = BDCAB; n = |Y| = 5

i	j	0	1	2	3	4	5
		Y_j	**B**	**D**	**C**	**A**	**B**
0	X_i		**0**	**0**	**0**	**0**	**0**
1	**A**	**0**					
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

for i = 1 to m c[i,0] = 0
for j = 1 to n c[0,j] = 0



if (X_i == Y_j)

c[i,j] = c[i-1,j-1] + 1

else c[i,j] = max(c[i-1,j], c[i,j-1])

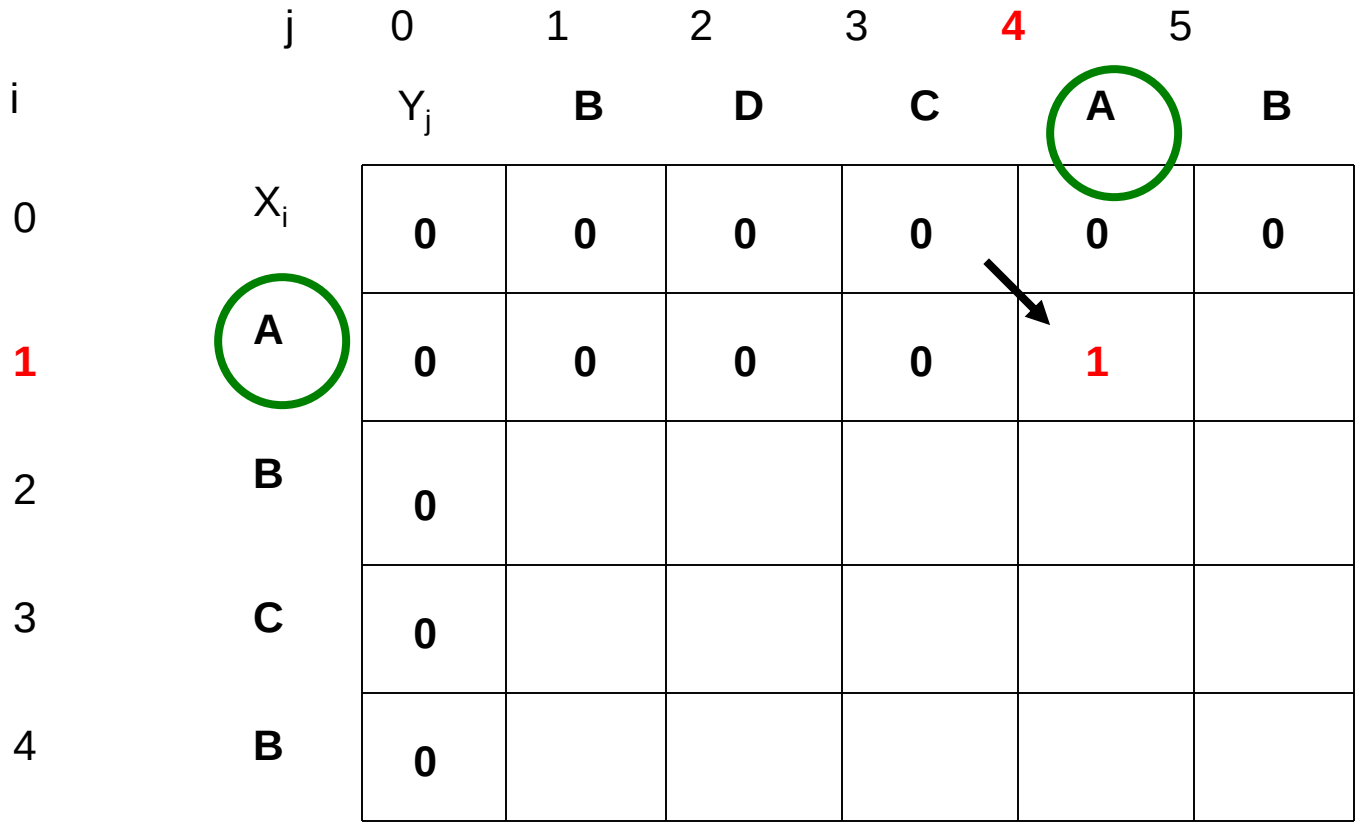
ABCB

BDCAB

i	j	0	1	2	3	4	5
		Y_j	**B**	**D**	**C**	**A**	**B**
0	X_i	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**		
2	**B**	**0**					
3	**C**	**0**					
4	**B**	**0**					

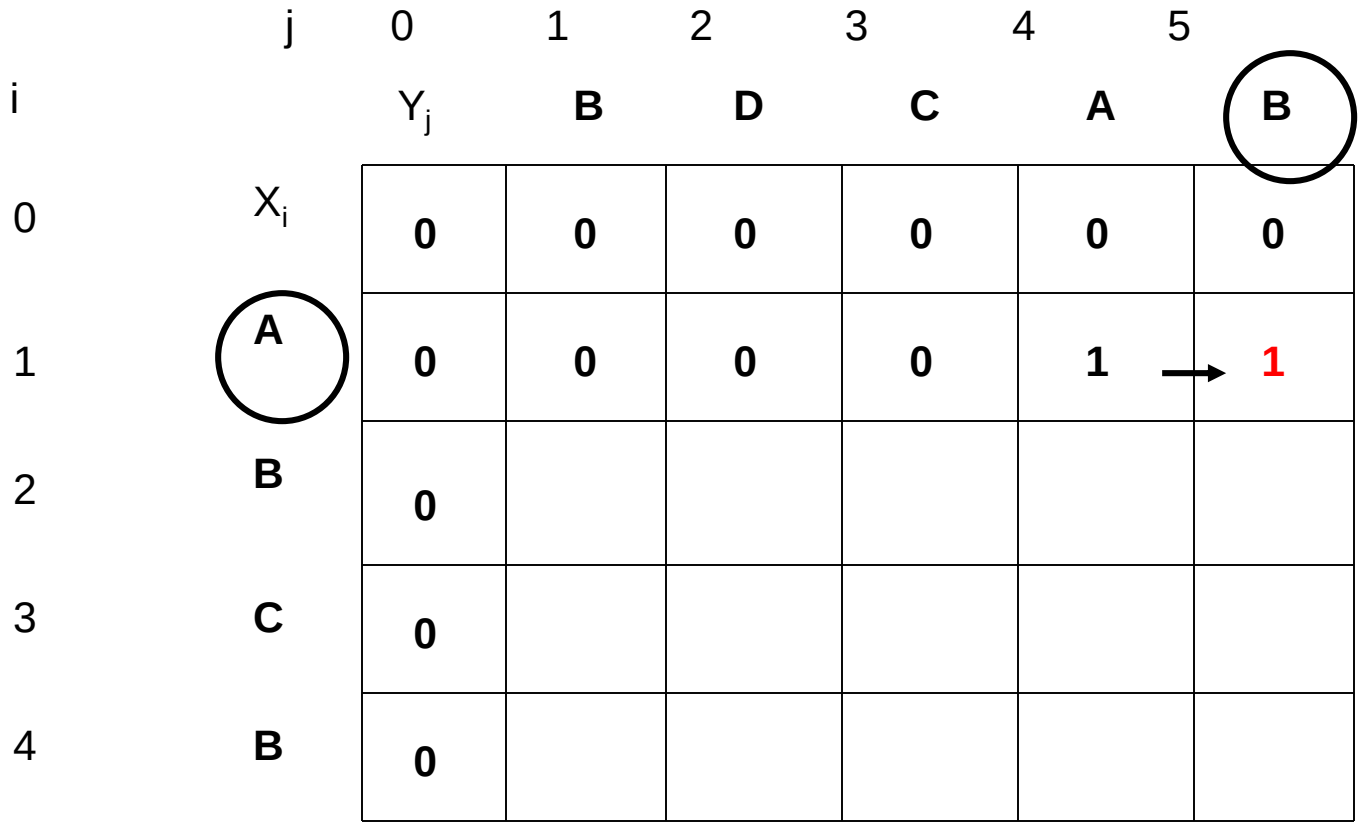
if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB



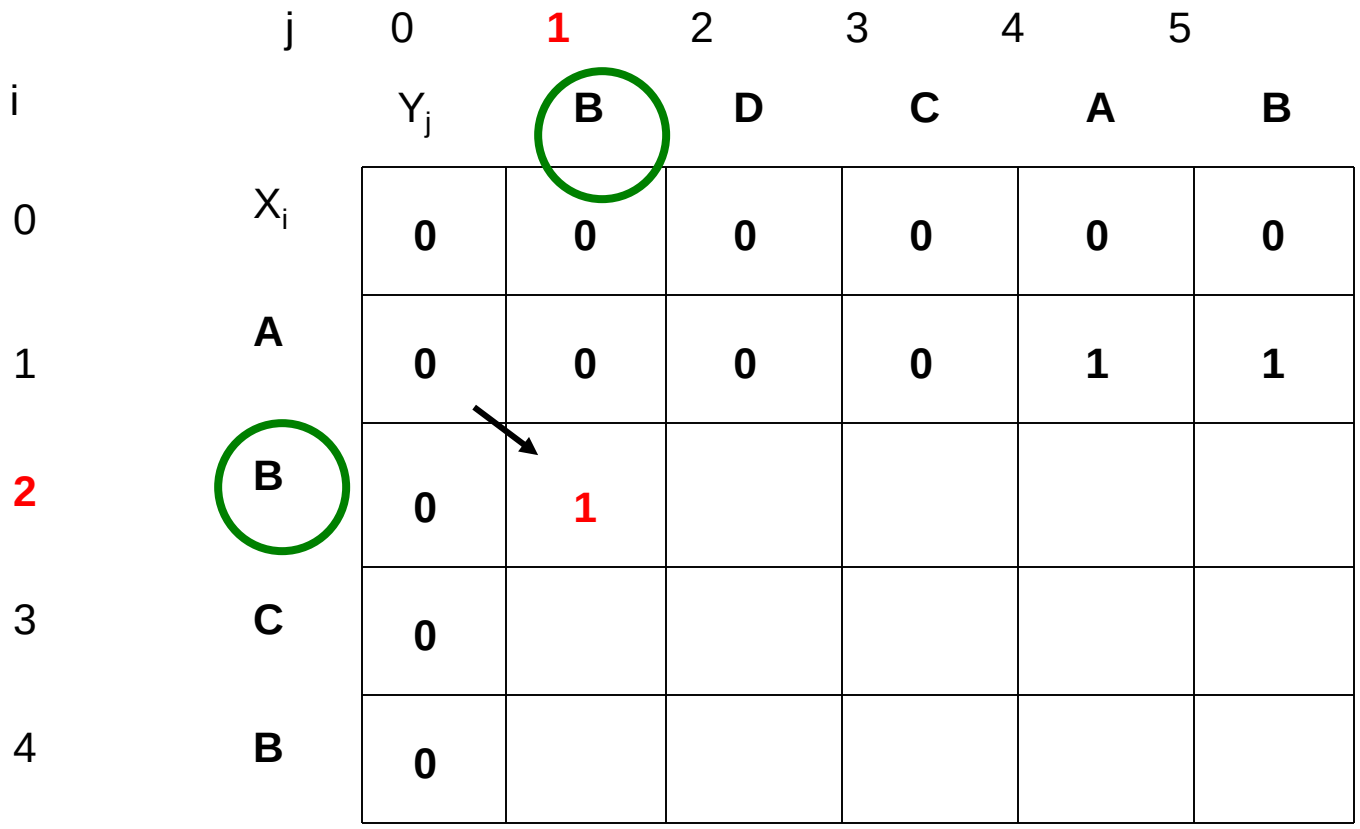
if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB



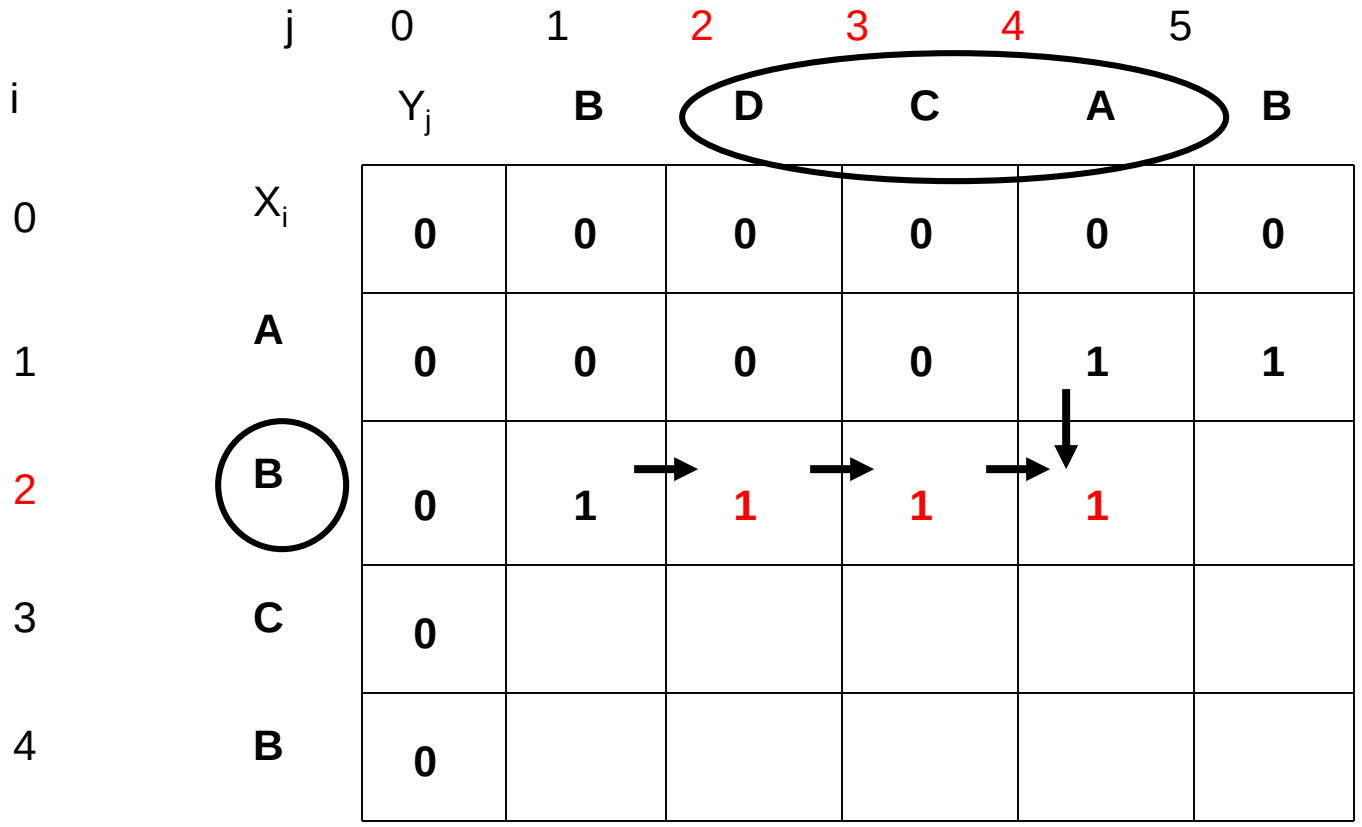
if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB



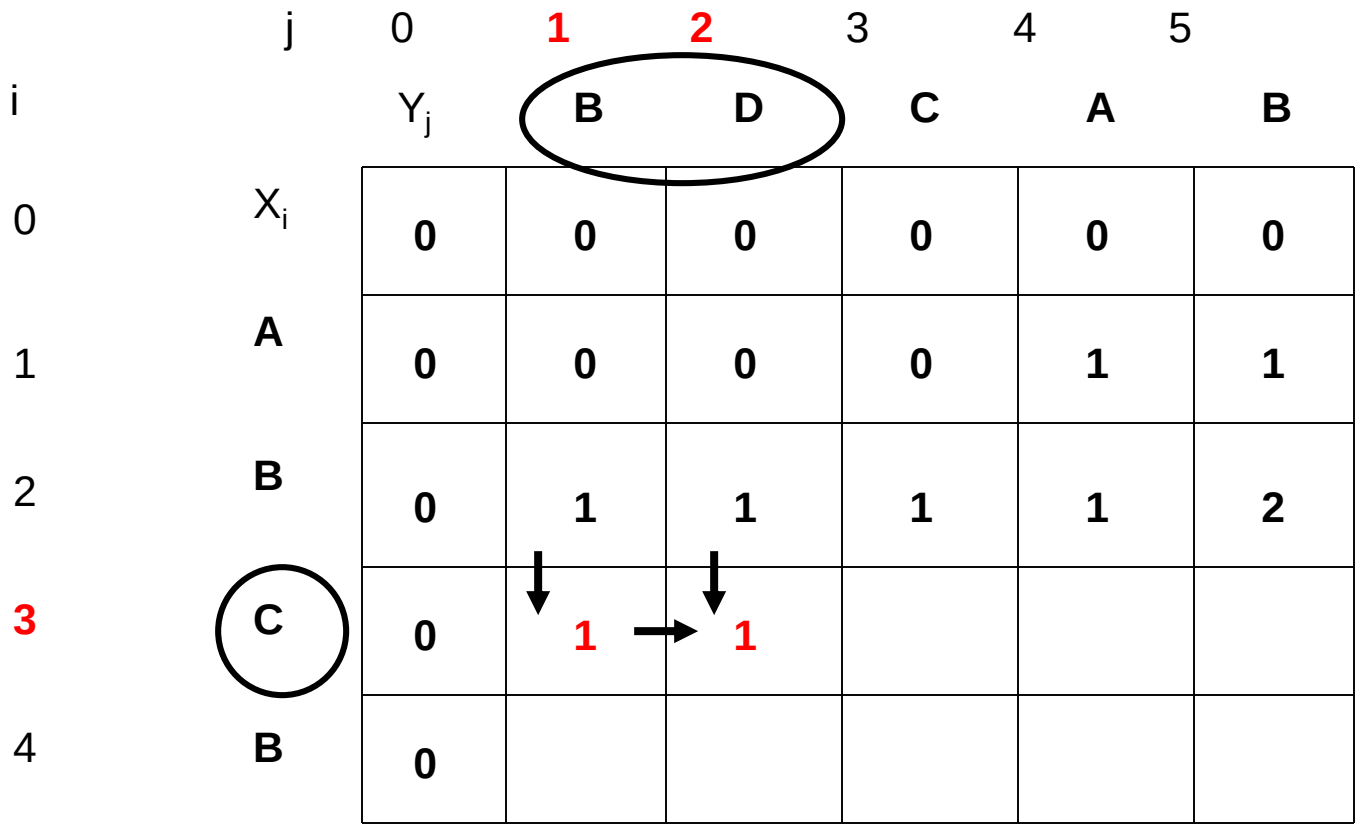
if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB

i		j	0	1	2	3	4	5
			Y_j	**B**	**D**	**C**	**A**	**B**
0	X_i		**0**	**0**	**0**	**0**	**0**	**0**
1	**A**		**0**	**0**	**0**	**0**	**1**	**1**
2	**B**		**0**	**1**	**1**	**1**	**1**	**2**
3	**C**		**0**					
4	**B**		**0**					

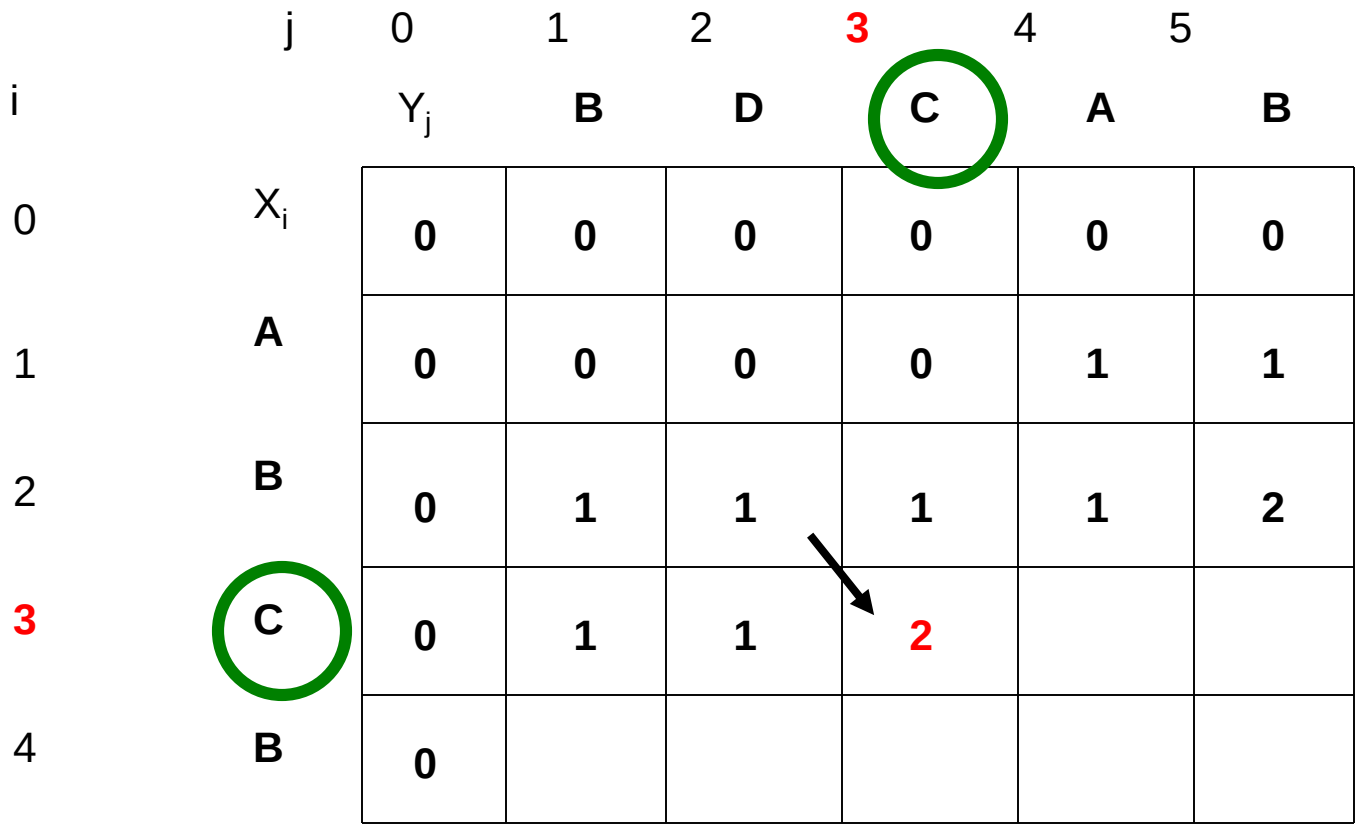
if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB
BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

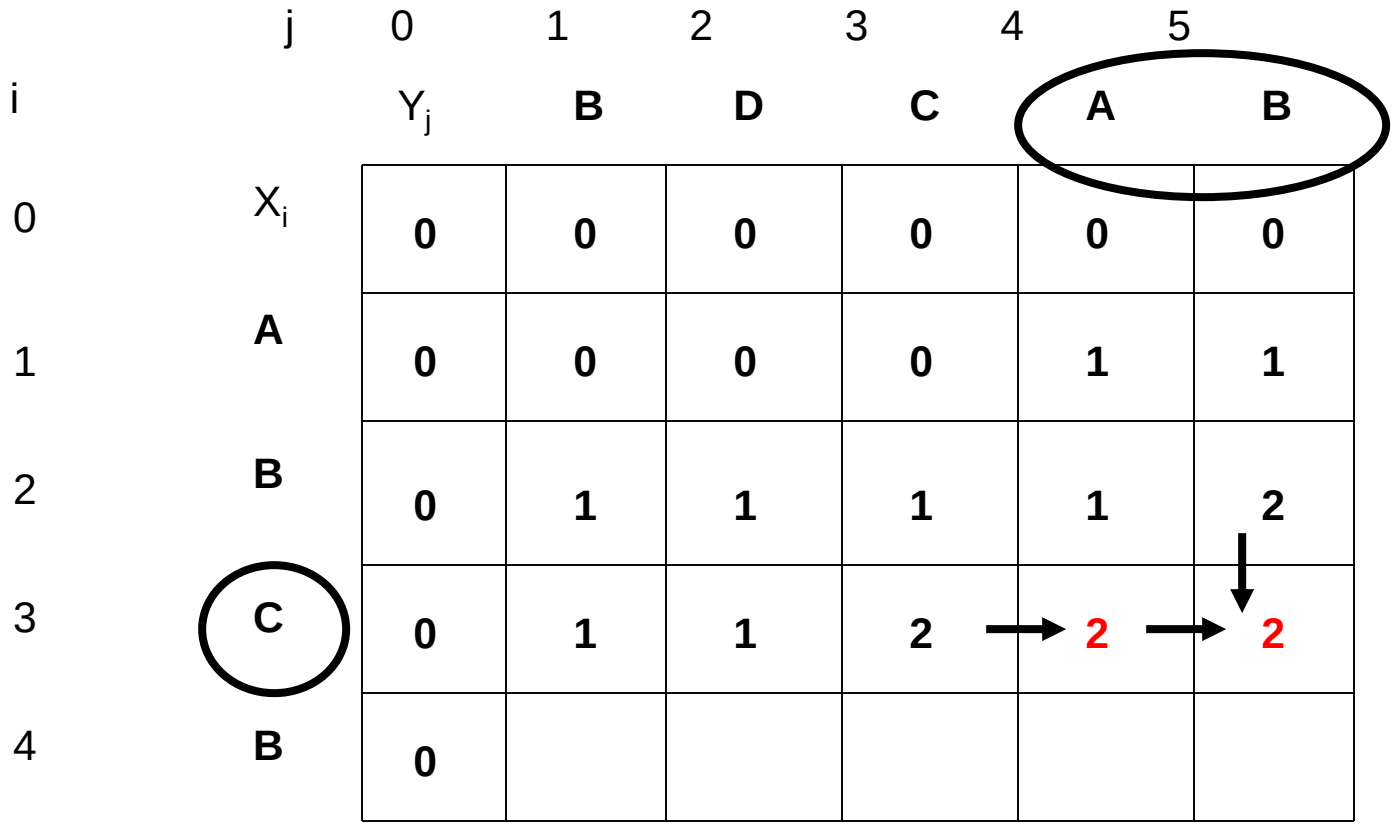
ABCB
BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB

BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB

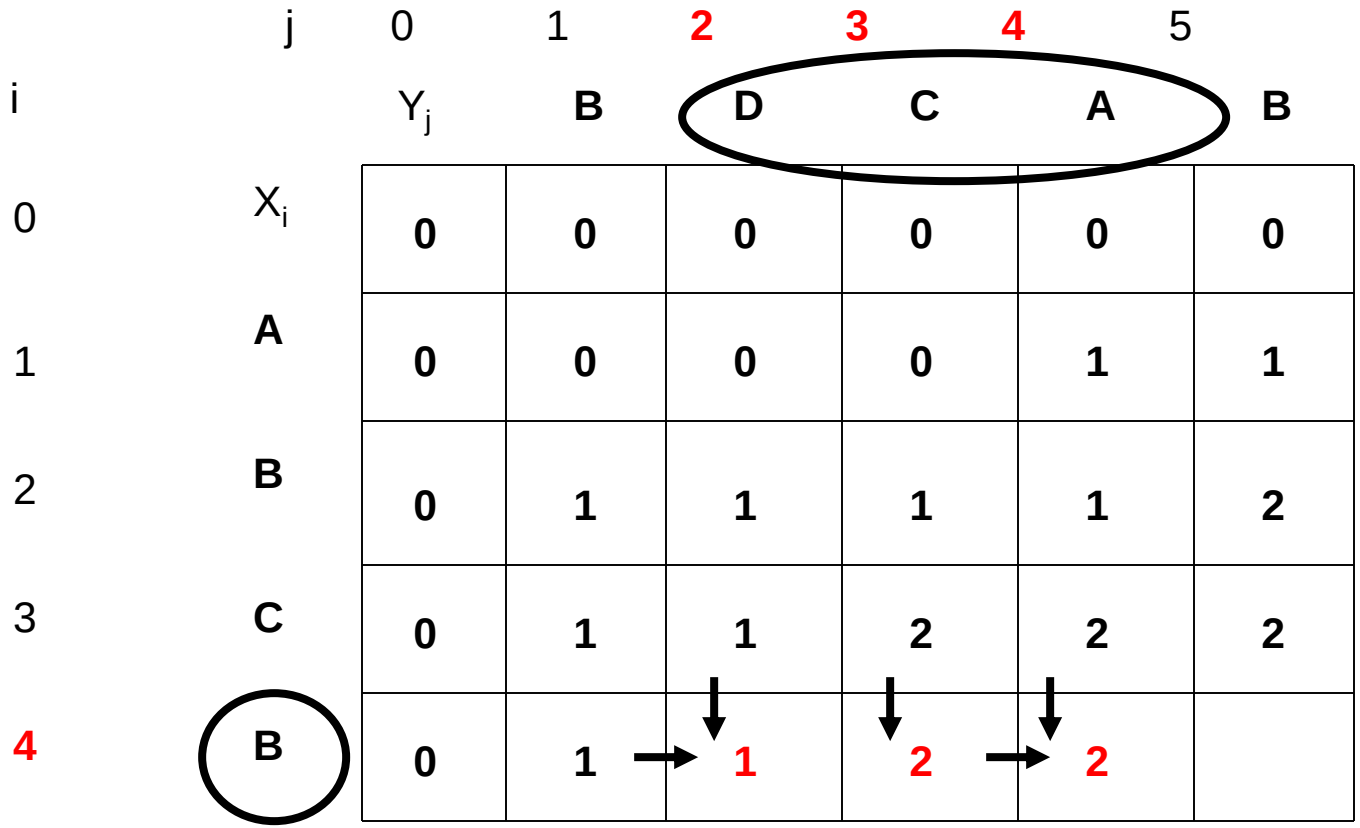
BDCAB

i	j	0	1	2	3	4	5
		Y_j	**B**	**D**	**C**	**A**	**B**
0	X_i	**0**	**0**	**0**	**0**	**0**	**0**
1	**A**	**0**	**0**	**0**	**0**	**1**	**1**
2	**B**	**0**	**1**	**1**	**1**	**1**	**2**
3	**C**	**0**	**1**	**1**	**2**	**2**	**2**
4	**B**	**0**	**1**				

```
if ( Xi == Yj )
                c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max( c[i-1,j], c[i,j-1] )
```

ABCB

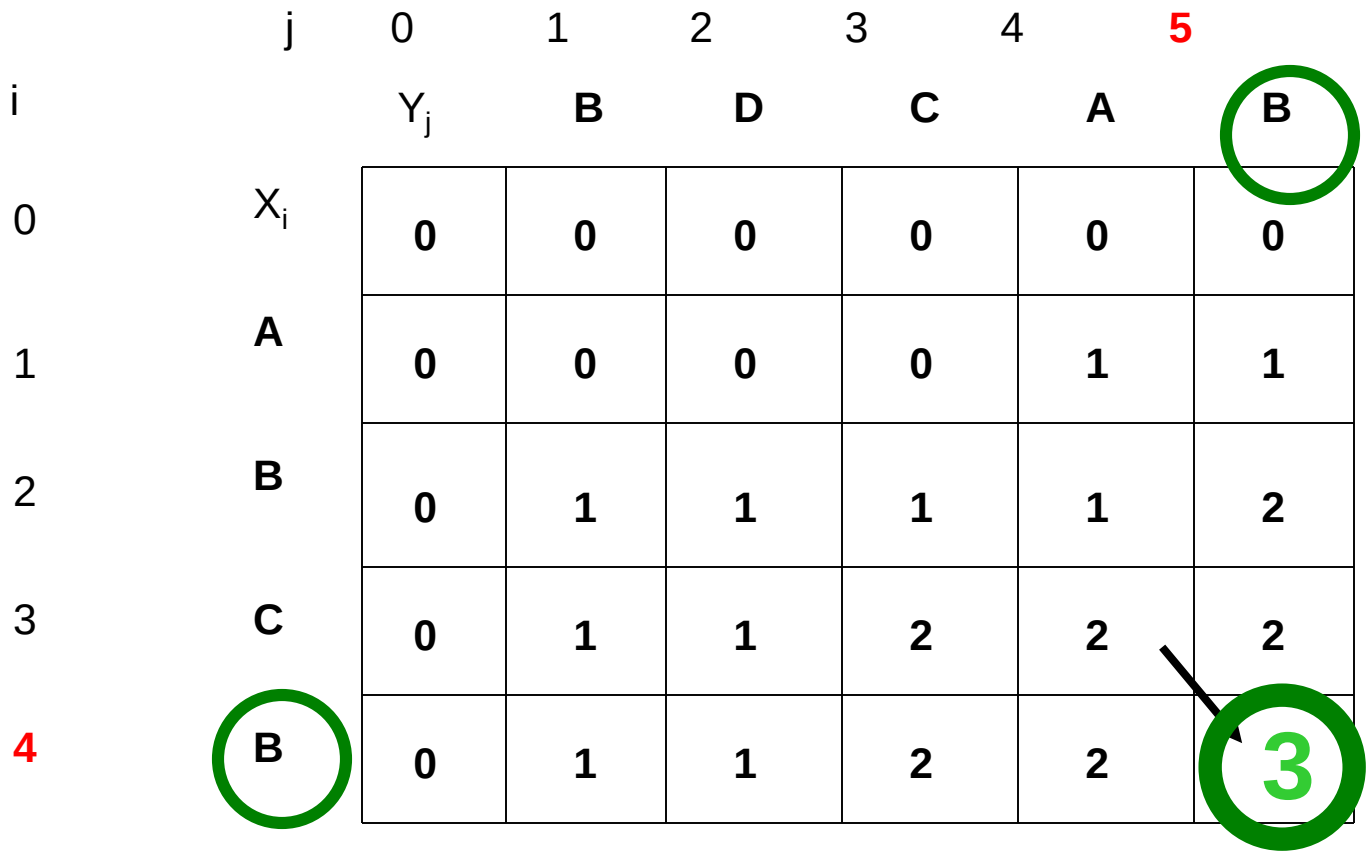
BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB

BDCAB



if (X_i == Y_j)
c[i,j] = c[i-1,j-1] + 1
else c[i,j] = max(c[i-1,j], c[i,j-1])

ABCB

BDCAB

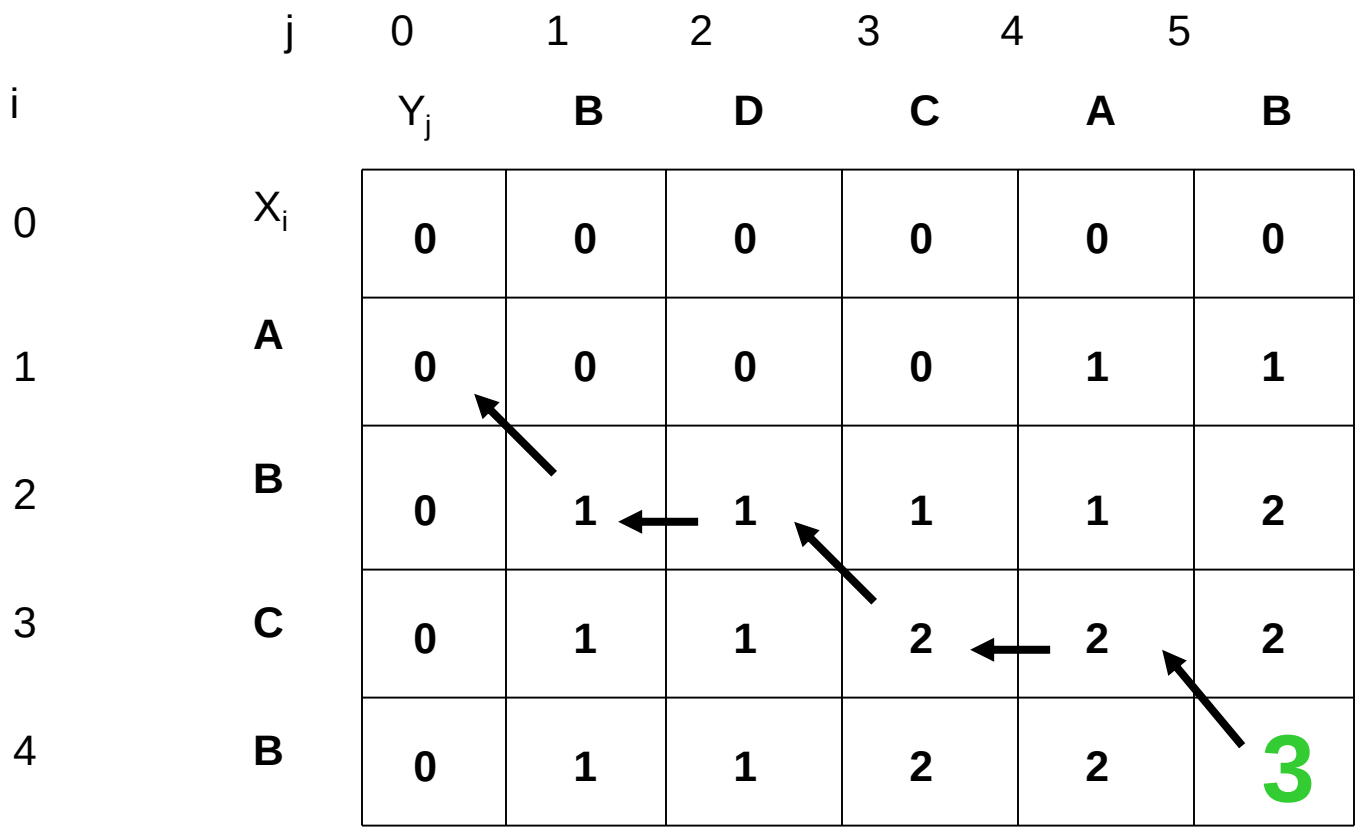
How to find actual LCS

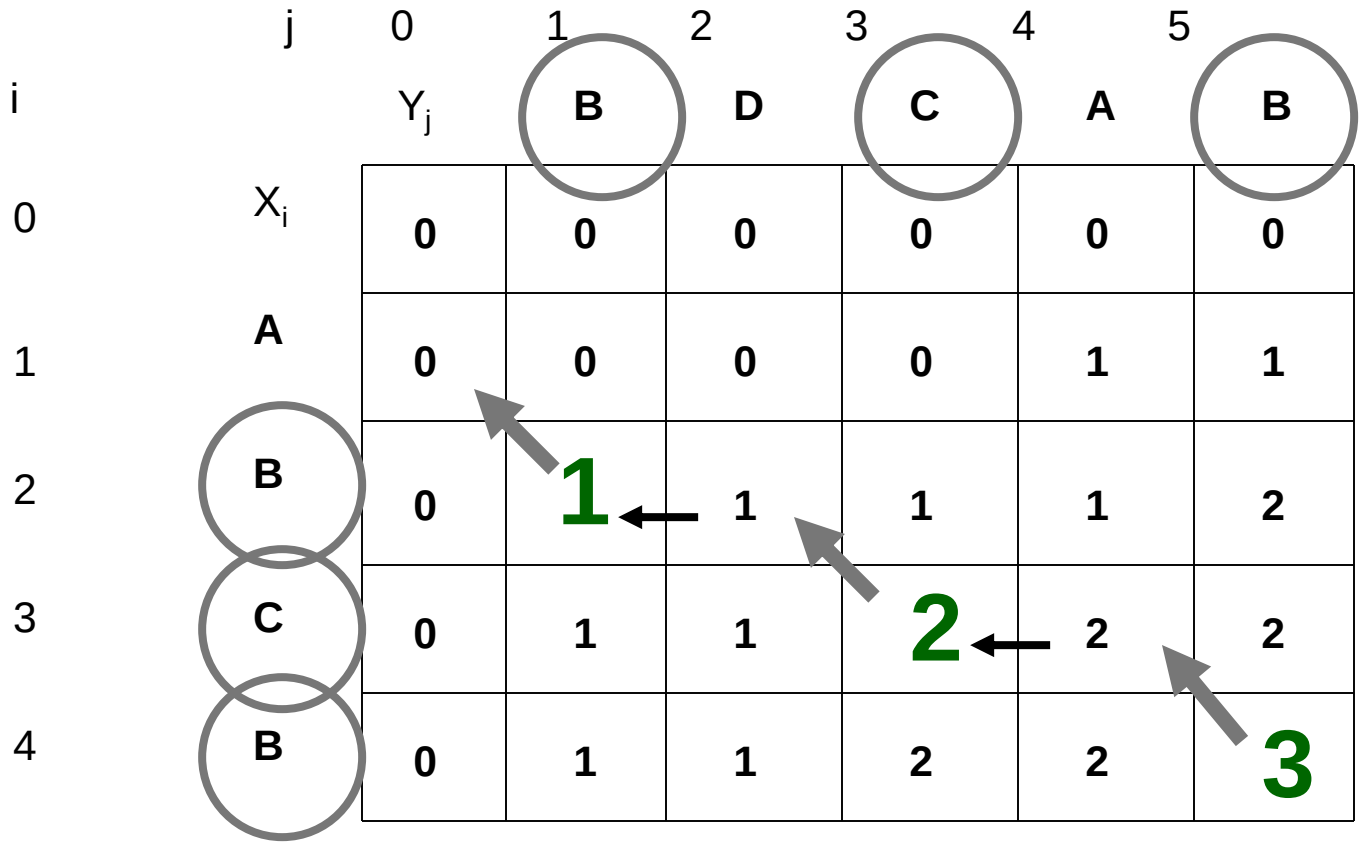
We have got the *length* of LCS, but not LCS itself.

To get actual LCS:

- Start from c[m,n] and go backwards
- Whenever c[i,j] = c[i-1, j-1]+1, remember X[i]
- When i=0 or j=0, output letters in reverse order

Finding LCS


j 0 1 2 3 4 5
i
Y_j B D C A B
0 X_i 0 0 0 0 0 0
1 A 0 0 0 0 1 1
2 B 0 1 1 1 1 2
3 C 0 1 1 2 2 2
4 B 0 1 1 2 2 3


j
0
1
2
3
4
5
i
Y_j
B
D
C
A
B
0
X_i
0
0
0
0
0
0
1
A
0
0
0
0
1
1
2
B
0
1
1
1
2
3
C
0
1
1
2
2
4
B
0
1
1
2
2
3

LCS: B C B

Knapsack problem

There are two versions of the problem:

1) 0/1 knapsack problem

Items are not divisible; you either take an item or not.

2) Fractional knapsack problem

Knapsack will be filled up by indivisible items as far as possible. If the knapsack can still store some weight, but the weights of other items exceed the limit, the fractional part of the next item can be added.

0/1 Knapsack Problem

Given: A set S of n items, with each item i having

b_i - a positive "benefit"

w_i - a positive "weight"

Goal: Choose items with maximum total benefit but with weight at most W.

Maximize Σb_i subject to $\Sigma w_i \leq W$

Example

Knapsack problem.

- Given n objects and a "knapsack".
- Item i weighs $w_i > 0$ kilograms and has value $b_i > 0$.
- Knapsack has capacity of W kilograms.
- Goal: fill knapsack so as to maximize total value.

Ex: { 3, 4 } has value 40.

W = 11

Item	Benefit	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

Greedy: repeatedly add item with maximum ratio b_i / w_i.

Ex: { 5, 2, 1 } achieves only value = 35 $\Rightarrow$ greedy not optimal.

Fractional Knapsack Problem

Given: A set S of n items, with each item i having

b_i - a positive benefit

w_i - a positive weight

Goal: Choose items with maximum total benefit but with weight at most W.

We are allowed to take fractional amounts.

- Let x_i denote the amount we take of item i
- Objective: maximize $\sum_{i \in S} b_i (x_i / w_i)$
- Constraint: $\sum_{i \in S} x_i \leq W$

The Fractional Knapsack Algorithm

```
Fractional-Knapsack(S, W)

for each item i in S
        x[i] ← 0
        v[i] ← b[i] / w[i]

sort items in decreasing order of v[i]
w ← 0

for each item i in S, in decreasing order of v[i]
   if (w + w[i]) ≤ W
        x[i] ← 1
        w ← w + w[i]
   else
        x[i] ← (W - w) / w[i]
        w ← W
        break
return x
```